

Proyecto Final

Jose Andres Roca Olivares 202501363

Tomas Ordoñez Ordoñez 202106247

*Escuela de Mecánica Eléctrica, Facultad de Ingeniería,
Universidad de San Carlos de Guatemala*

I. Resumen

El presente proyecto consiste en el desarrollo de una aplicación de escritorio orientada a la gestión básica de una empresa eléctrica, elaborada en Python. El sistema permite registrar información de clientes, calcular automáticamente el consumo energético y generar facturas digitales en formato PDF.

La aplicación inicia creando automáticamente las carpetas necesarias para el almacenamiento de información y facturas. Posteriormente, el usuario accede mediante un sistema de inicio de sesión que brinda seguridad al sistema a través del manejo de credenciales. Una vez dentro, se presenta un formulario donde se ingresan los datos del cliente, como nombre, NIT, contador y consumo eléctrico en kilovatios por hora.

El sistema realiza automáticamente el cálculo del costo del servicio eléctrico utilizando una tarifa fija de Q1.42 por kWh, además de aplicar el IVA correspondiente para obtener el total a pagar. Posteriormente, se genera una factura profesional en formato PDF utilizando la librería FPDF, permitiendo almacenar y visualizar la información de manera ordenada.

Como funcionalidad adicional, el sistema incorpora el envío automático de facturas al correo electrónico del cliente mediante el protocolo SMTP, adjuntando el archivo PDF generado. Asimismo, el proyecto integra herramientas de seguridad como cifrado de contraseñas con SHA-256, validación de contraseñas seguras, control de intentos fallidos y registro de actividades en una bitácora, fortaleciendo la confiabilidad y protección de la información manejada por el sistema.

II. Objetivos

Desarrollar una aplicación de escritorio en Python para la gestión básica de una empresa eléctrica, que permita registrar datos de clientes, calcular el consumo energético, generar facturas en formato PDF y enviarlas por correo electrónico de manera automatizada.

III. Marco Teórico

Python es un lenguaje de programación de alto nivel ampliamente utilizado en el desarrollo de aplicaciones de escritorio, automatización de procesos y manejo de datos. Se caracteriza por su sintaxis sencilla y fácil comprensión, lo que facilita el desarrollo de proyectos informáticos. En este proyecto, Python fue utilizado como lenguaje principal para la creación de toda la lógica del sistema de gestión de la empresa eléctrica.

Tkinter es una librería gráfica incluida en Python que permite desarrollar interfaces visuales mediante ventanas, botones, etiquetas y cuadros de texto. Gracias a esta herramienta fue posible crear la interfaz gráfica del sistema, facilitando la interacción del usuario con las diferentes funciones del programa, como el inicio de sesión, el registro de datos y la generación de facturas.

FPDF es una librería utilizada para la creación de archivos

PDF en Python. Esta herramienta permite generar documentos personalizados de manera automática, agregando texto, tablas y diferentes formatos. En el proyecto se utilizó para generar las facturas eléctricas en formato PDF de manera organizada y profesional.

SMTP (Simple Mail Transfer Protocol) es un protocolo utilizado para el envío de correos electrónicos a través de internet. En este sistema se implementó para permitir el envío automático de facturas al correo electrónico del cliente, adjuntando el archivo PDF generado por el programa.

SHA-256 es un algoritmo criptográfico de tipo hash que transforma una contraseña en una cadena de caracteres cifrada e irreversible. Su función principal es proteger la información sensible evitando almacenar contraseñas en texto plano. En el proyecto se utilizó para mejorar la seguridad del sistema de inicio de sesión.

JSON (JavaScript Object Notation) es un formato ligero de almacenamiento e intercambio de datos. Se utiliza para guardar información de manera estructurada y fácil de leer. En el sistema fue implementado para almacenar la configuración del correo electrónico utilizado para el envío automático de facturas.

IV. Marco Practico

Inicialmente se desarrolló la interfaz de inicio de sesión y otra para que el usuario pueda poner datos del cliente utilizando la librería Tkinter en Python. Se diseñó una ventana un botón, etiquetas y cuadros de texto que permiten al usuario interactuar de manera sencilla con el sistema.

INICIO DE SESIÓN

Usuario

Contraseña

Ingresar

Correlativo	
Nombre	
NIT	
Contador	
Consumo (kWh)	
Correo	

[Generar y Enviar Factura](#)

Después se creó un panel de administración, donde era posible registrar nuevos usuarios y eliminar usuarios

Sistema Empresa Eléctrica

Administrador

Usuario
 Contraseña
 Confirmar

Registrar
 Eliminar Usuario

Usuario	Hash
admin	03ac67421ff3e15c761ee1a5c259067e
admin23	83e19a9ce479dc064b4b4bd50134db1
Tomas	cbfad02f9ed2ad81e0bd9f74f5300e9ei
Arthur	884787790b634bcded4b97eb6d7462ff(
	e3b0c44298fc1c148afb4c8999fe9242

Facturación
 Bitácora
 Cerrar sesión

Ya con una base establecida se ordenó la interfaz, se implementó la librería FPDF para poder generar facturas en formato pdf, la configuración y envíos de correos electrónicos y la posibilidad de que el usuario pueda cambiar su contraseña

V. Código

```

1 from sys import sys
2 import hashlib
3 import re
4 import smtplib
5 import json
6 from tkinter import *
7 from tkinter import messagebox, ttk
8 from datetime import datetime
9 from http import http
10 from email.mime.multipart import MIMEMultipart
11 from email.mime.text import MIMEText
12 from email.mime.base import MIMEBase
13 from email import encoders
14
15 # --- CONFIGURACIÓN Y COLORES ---
16 COLOR_BG = "#e0e0e0"
17 COLOR_SECUNDARIO = "#444444"
18
19 # --- FUNCIONES DE SEGURIDAD ---
20 def encryptar(password):
21     return hashlib.sha256(password.encode()).hexdigest()
22
23 def validar_password(separa(password)):
24     """Verifica que la contraseña cumpla con estándares de seguridad."""
25     if len(password) < 8:
26         return False, "La contraseña debe tener al menos 8 caracteres."
27     if not re.search("[a-z]", password):
28         return False, "Debe incluir al menos una letra minúscula."
29     if not re.search("[0-9]", password):
30         return False, "Debe incluir al menos un número."
31     if not re.search("[!@#$%^&*()_+]", password):
32         return False, "Debe incluir al menos un carácter especial (!, @, #, etc.)."
33     return True, ""
34
35 def mostrar_mensaje(mensaje):

```

```

40 def cargar_config_correo(self, f):
41     json.dump(datos, f)
42
43 def cargar_config_correo(self):
44     if os.path.exists("Configuracion/correo.json"):
45         with open("Configuracion/correo.json", "r") as f:
46             return json.load(f)
47         return ("remiteinte": "", "password": "")
48
49 # --- GESTIÓN DE ARCHIVOS ---
50 def crear_directorios():
51     for carpeta in ["Configuracion", "Facturas"]:
52         if not os.path.exists(carpeta):
53             os.makedirs(carpeta)
54
55 def registrar_bitacora(action, usuario="Sistema"):
56     fecha = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
57     with open("Configuracion/bitacora.txt", "a") as f:
58         f.write(f"{fecha}|{usuario}|{action}\n")
59
60 def obtener_intentos():
61     intentos = []
62     if os.path.exists("Configuracion/intentos.txt"):
63         with open("Configuracion/intentos.txt") as f:
64             for linea in f:
65                 u, l = linea.strip().split(",")
66                 intentos[u] = int(l)
67             except: continue
68     return intentos
69
70 def guardar_intentos(intentos):
71     with open("Configuracion/intentos.txt", "w") as f:
72         for u, l in intentos.items():
73             f.write(f"{u},{l}\n")
74
75 # --- CLASE PRINCIPAL ---
76 class App:
77     def __init__(self, root):
78         self.root = root
79         self.root.title("Sistema Empresa Eléctrica")
80         self.root.geometry("800x500")
81         self.root.config(bg="#f0f0f0")
82         self.usuario_actual = None
83         self.login()
84
85 def clear(self):
86     for w in self.root.winfo_children():
87         w.destroy()
88
89 def header(self, text):
90     Label(self.root, text=text, bg="#f0f0f0", font=("Arial", 16, "bold"), pack(fill="x", pady=10))
91
92 # --- LOGIN ---
93 def login(self):
94     self.clear()
95     self.header("Inicio de Sesión")
96     f = Frame(self.root, bg="white", padx=10, pady=10)
97     f.pack(pady=50)
98     Label(f, text="Usuario:", bg="white").pack()
99     self.ent_user = Entry(f)
100     self.ent_user.pack()
101     Label(f, text="Contraseña:", bg="white").pack()
102     self.ent_pass = Entry(f, show="*")
103     self.ent_pass.pack()
104     Button(f, text="Ingresar", command=self.validar_acceso).pack(pady=10)
105
106 def obtener_usuarios(self):
107     usuarios = []
108     if os.path.exists("Configuracion/usuarios.txt"):
109         with open("Configuracion/usuarios.txt") as f:
110             for linea in f:
111                 u, p = linea.strip().split(",")
112                 usuarios[u] = p
113     return usuarios
114
115 def validar_acceso(self):
116     user = self.ent_user.get()
117     pas = self.ent_pass.get()
118     db = self.obtener_usuarios()
119     intentos = obtener_intentos()
120
121     if user in db and db[user] == encryptar(pas) or db[user] == pas:
122         messagebox.showerror("Bloqueado", "Usuario bloqueado por intentos fallidos")
123         return
124
125     if user in db and (db[user] == encryptar(pas) or db[user] == pas):
126         self.usuario_actual = user
127         registrar_bitacora("Inicio de sesión", user)
128         intentos[user] = 0
129         guardar_intentos(intentos)
130         if user in ["admin", "admin2"]:
131             self.modo_administrador()
132         else:
133             self.mnu_facturacion()
134         return
135
136     intentos[user] = intentos.get(user, 0) + 1
137     guardar_intentos(intentos)
138     messagebox.showerror("Error", "Datos incorrectos")
139
140 # --- ADMINISTRACIÓN ---
141 def modo_administrador(self):
142     self.clear()
143     self.header("Panel Administrador")
144
145 # Datos Configuración de Correo
146 Button(self.root, text="Configurar Servidor de Correo",
147        command=self.config_correo_window, bg="#e0e0e0", fg="white").pack(pady=5)
148
149 # Borrar Configuración de Correo
150 Button(self.root, text="Borrar credenciales de Correo",
151        command=self.borrar_config_correo, bg="#e0e0e0", fg="white").pack(pady=5)
152
153 f = Frame(self.root, bg="white", padx=20, pady=20)
154 f.pack()
155 self.mnu_u = Entry(f)
156 self.mnu_u.grid(row=0, column=0)
157 self.mnu_r = Entry(f)
158 self.mnu_r.grid(row=1, column=0)
159 self.mnu_p = Entry(f, show="*")
160 self.mnu_p.grid(row=2, column=0)
161 self.mnu_c = Entry(f, show="*")
162 self.mnu_c.grid(row=3, column=0)
163 self.confirmp = Entry(f, show="*")
164 self.confirmp.grid(row=4, column=0)
165 Button(f, text="Registrar Usuario", command=self.guardar_usuario).grid(row=5, columnspan=2, pady=10)
166
167 self.tabla = ttk.Treeview(self.root, columns=("User", "Hash"), show="headings")
168 self.tabla.heading("User", text="Usuario")
169 self.tabla.heading("Hash", text="Hash (Seguridad)")
170 self.tabla.pack(pady=10)
171 self.actualizar_tabla()
172
173 Button(self.root, text="Eliminar Usuario Seleccionado", command=self.eliminar_usuario).pack()
174 Button(self.root, text="Ir a facturación", command=self.mnu_facturacion).pack(pady=5)
175 Button(self.root, text="Cerrar sesión", command=self.logout).pack()
176
177 def config_correo_window(self):
178     win = Toplevel(self.root)
179     win.title("Configuración de Correo")
180     win.geometry("600x300")
181
182     config = cargar_config_correo()
183
184     Label(win, text="Correo Remiteinte (email)", fg="white").pack(pady=5)
185     ent_rem = Entry(win, width=40)
186     ent_rem.insert(0, config["remiteinte"])
187     ent_rem.pack()
188
189     Label(win, text="Contraseña de Aplicación (16 caracteres)", fg="white").pack(pady=5)
190     ent_pas = Entry(win, width=40, show="*")
191     ent_pas.insert(0, config["password"])
192     ent_pas.pack()
193
194 def guardar():
195     guardar_config_correo("remiteinte", ent_rem.get(), "password", ent_pas.get())
196     messagebox.showinfo("Éxito", "Configuración de correo guardada")
197     win.destroy()
198
199 Button(win, text="Guardar Configuración", command=guardar, bg="#e0e0e0", fg="white").pack(pady=20)
200
201 def borrar_config_correo(self):
202     # Confirmamos con el usuario para evitar borrados accidentales
203     confirmar = messagebox.askyesno("Confirmar Borrado",
204                                     "¿Estás seguro de que deseas eliminar las credenciales del correo?")
205     if confirmar:
206         # Desactivamos el envío de facturas hasta que se configure de nuevo.
207         if confirmar:
208             # Sobrescribimos el archivo con datos vacíos
209             guardar_config_correo("remiteinte": "", "password": "")
210
211     registrar_bitacora("Configuración de correo eliminada", self.usuario_actual)
212     messagebox.showinfo("Éxito", "Credenciales eliminadas correctamente.")
213
214 def guardar_usuario(self):
215     self.mnu_u.get()
216     p = self.mnu_p.get()
217     cp = self.confirmp.get()
218     if p != cp:

```

```

216         return
217     es_segura, mensaje = validar_password_segura(p)
218     if not es_segura:
219         messagebox.showwarning("Seguridad", mensaje)
220     return
221     with open("configuracion/usuarios.txt", "a") as f:
222         f.write(f"({u}),{encryptar(p)}\n")
223     registrar_bitacora("Usuario creado: {u}", self.usuario_actual)
224     self.actualizar_tabla()
225     messagebox.showinfo("Éxito", "Usuario registrado")
226
227     def actualizar_tabla(self):
228         selection = self.tabla.selection()
229         for i in self.tabla.get_children(): self.tabla.delete(i)
230         for u, p in self.obtener_usuarios().items():
231             self.tabla.insert("", END, values=(u, p))
232
233     def eliminar_usuario(self):
234         selection = self.tabla.selection()
235         if not selection: return
236         usuario = self.tabla.item(selection[0])["values"][0]
237         if usuario == "admin":
238             messagebox.showerror("Error", "No se puede eliminar al admin")
239         return
240         usuarios = self.obtener_usuarios()
241         del usuarios[usuario]
242         with open("configuracion/usuarios.txt", "w") as f:
243             for u, p in usuarios.items(): f.write(f"({u}),{p}\n")
244         self.actualizar_tabla()
245
246     # ---- FACTURACIÓN ----
247     def menu_facturacion(self):
248         self.clear()
249         # Marco de herramientas superior
250         nav = Frame(self.root, bg=COLOR_NAV)
251         nav.grid(row=0, column=0, sticky="nsew")
252
253     # Configuración de Usuario, comando=self.cambiar_password_window.pack(side="left")
254     self.header("Módulo de Facturación")
255     f = Frame(self.root, bg="white", padx=20, pady=20)
256     f.pack(pady=20)
257
258     campos = [
259         ("Correlativo", "correlativo", "Nombre", "nombre"),
260         ("NIT", "nit", "Contador", "contador"), ("Consumo kWh", "consumo"),
261         ("Correo Cliente", "correo")
262     ]
263     self.entradas = {}
264     for i, (texto, var) in enumerate(campos):
265         Label(f, text=texto, bg="white").grid(row=i, column=0, padx=10, pady=5, sticky="n")
266         e = Entry(f, width=30)
267         e.grid(row=i, column=1, pady=5)
268         self.entradas[var] = e
269
270     Button(f, text="Generar y Enviar Factura", command=self.generar_factura,
271           bg="#f2f2f2", fg="white", font=("Arial", 10, "bold")).grid(row=len(campos), columnspan=2, pady=20)
272     Button(self.root, text="Cerrar Sesión / Volver", command=self.login.pack())
273
274     # ---- CONFIGURACIÓN DE USUARIO (CAMBIAR PASS) ----
275     def cambiar_password_window(self):
276         win = Toplevel(self.root)
277         win.title("Cambiar Contraseña")
278         win.geometry("400x250")
279         win.grab_set()
280
281         Label(win, text="Actualizar Seguridad", font=("Arial", 12, "bold")).pack(pady=10)
282
283         Label(win, text="Contraseña Actual").pack()
284         ent_act = Entry(win, show="*")
285         ent_act.pack()
286
287         Label(win, text="Nueva Contraseña").pack(pady=(10,0))
288         ent_n1 = Entry(win, show="*")
289         ent_n1.pack()
290
291         Label(win, text="Confirmar Nueva Contraseña").pack()
292         ent_n2 = Entry(win, show="*")
293         ent_n2.pack()
294
295         def confirmar_cambio():
296             usuarios = self.obtener_usuarios()
297             if encryptar(ent_act.get()) != usuarios[self.usuario_actual]:
298                 messagebox.showerror("Error", "La contraseña actual no es correcta")
299                 return
300             if ent_n1.get() != ent_n2.get():
301                 messagebox.showerror("Error", "Las nuevas contraseñas no coinciden")
302                 return
303             es_segura, mensaje = validar_password_segura(ent_n1.get())
304             if not es_segura:
305                 messagebox.showwarning("Seguridad", mensaje)
306                 return
307             usuarios[self.usuario_actual] = encryptar(ent_n1.get())
308             for u, p in usuarios.items(): f.write(f"({u}),{p}\n")
309             registrar_bitacora("Cambio de contraseña", self.usuario_actual)
310             messagebox.showinfo("Éxito", "Contraseña cambiada correctamente")
311             win.destroy()
312
313         Button(win, text="Guardar Cambios", command=confirmar_cambio, bg=COLOR_SECONDARIO, fg="white").pack(pady=20)
314
315     # ---- ENVÍO DE CORREO ----
316     def enviar_correo(self, destinatario, ruta_pdf, corr):
317         config = cargar_config_correo()
318         if not config['remitente'] or not config['password']:
319             messagebox.showerror("Configuración fallida", "El admin no ha configurado el correo remitente.")
320         return False
321
322         try:
323             mg = MIMETextPart()
324             mg['from'] = config['remitente']
325             mg['to'] = destinatario
326             mg['subject'] = f"Factura Eléctrica No. {corr}"
327             mg.attach(MIMEText(f"Adjunto encontrará su factura No. {corr}. Saludos.", 'plain'))
328
329             with open(ruta_pdf, "rb") as f:
330                 parte = MIMEBase('application', 'octet-stream')
331                 parte.set_payload(f.read())
332                 encoders.encode_base64(parte)
333                 parte.add_header('Content-Disposition', f'attachment; filename=factura_{corr}.pdf')
334             mg.attach(parte)
335
336             server = smtplib.SMTP('smtp.gmail.com', 587)
337             server.starttls()
338             server.login(config['remitente'], config['password'])
339             server.sendmail(config['remitente'], destinatario, mg.as_string())
340             server.quit()
341             return True
342         except Exception as e:
343             print(f"Error: {e}")
344             return False
345
346     # ---- GENERACIÓN DE FACTURA ----
347     def generar_factura(self):
348         # Obtener datos
349         corr = self.entradas['correlativo'].get().strip()
350         nom = self.entradas['nombre'].get().strip()
351         nit = self.entradas['nit'].get().strip()
352         cont = self.entradas['contador'].get().strip()
353         cons_raw = self.entradas['consumo'].get().strip()
354         email_dest = self.entradas['correo'].get().strip()
355
356         # Validaciones con Regexp
357         if not re.match("^[0-9]+$", corr):
358             messagebox.showerror("Error", "Correlativo: solo números enteros positivos")
359             return
360         if not re.match("^[a-zA-ZÁÉÍÓÚÑ]{4}$", nom):
361             messagebox.showerror("Error", "Nombre: solo letras y espacios")
362             return
363         if not re.match("^[cf][cf][0-9]+$", nit):
364             messagebox.showerror("Error", "NIT: solo números o 'C'")
365             return
366         if not re.match("^[a-zA-Z0-9]{4}$", cont):
367             messagebox.showerror("Error", "Contador: solo letras y números")
368             return
369         if not re.match("^[0-9]+$", cons_raw):
370             messagebox.showerror("Error", "Consumo: solo números enteros")
371             return
372
373         # Cálculos
374         consumo = float(cons_raw)
375         precio_kwh = 1.42
376         subtotal = consumo * precio_kwh
377         iva = subtotal * 0.12
378         total = subtotal + iva
379
380         # Crear PDF
381         pdf = FPDF()
382         pdf.add_page()
383         pdf.set_font("Arial", "B", 10)
384         pdf.cell(0, 10, "EMPRESA ELÉCTRICA S.A.", 0, 1, "C")
385
386         pdf.set_font("Arial", "", 12)
387         pdf.cell(0, 5, f"Factura No. {corr}", 0, 1, "C")
388         pdf.cell(0, 5, f"Fecha: {datetime.now().strftime('%d/%m/%Y %H:%M')}", 0, 1, "C")
389         pdf.ln(10)

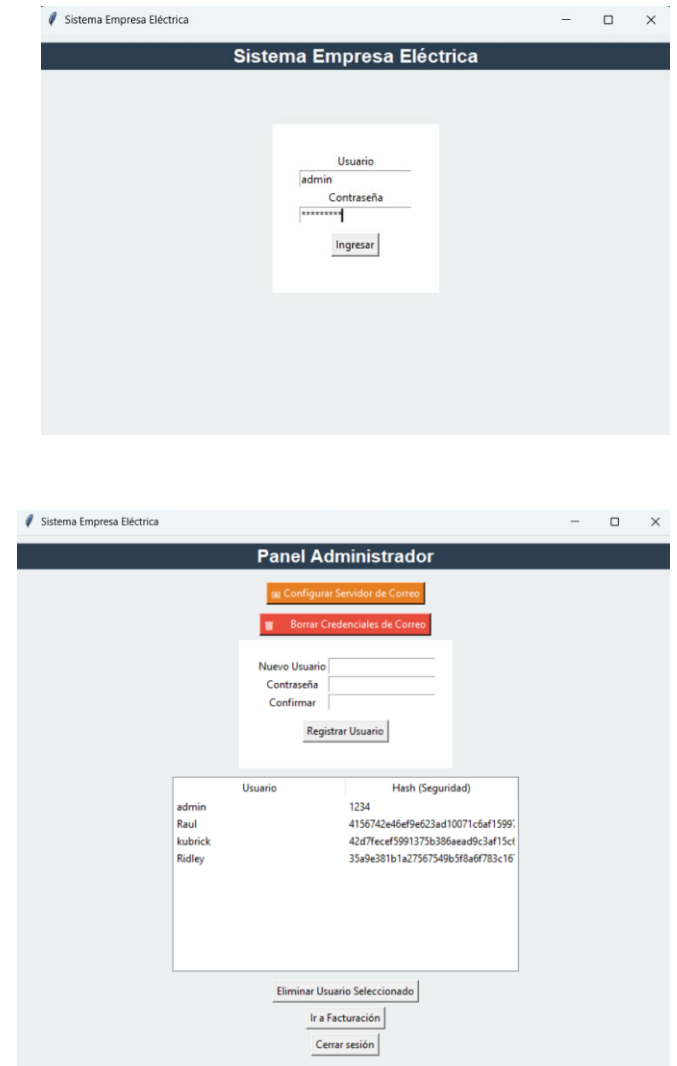
```

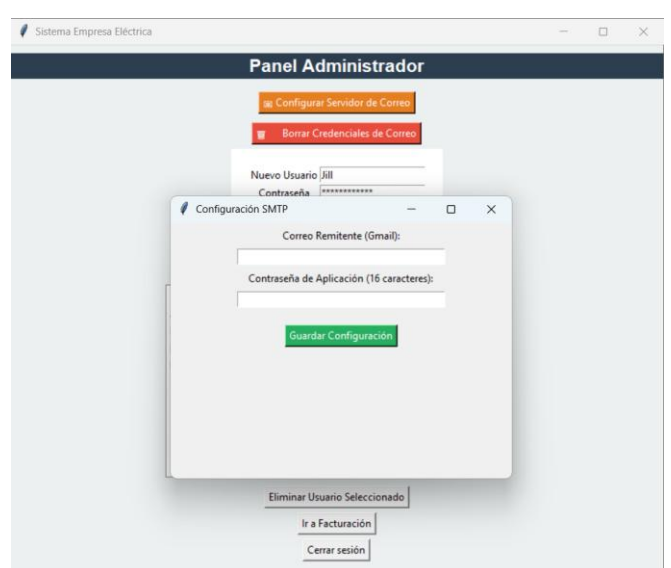
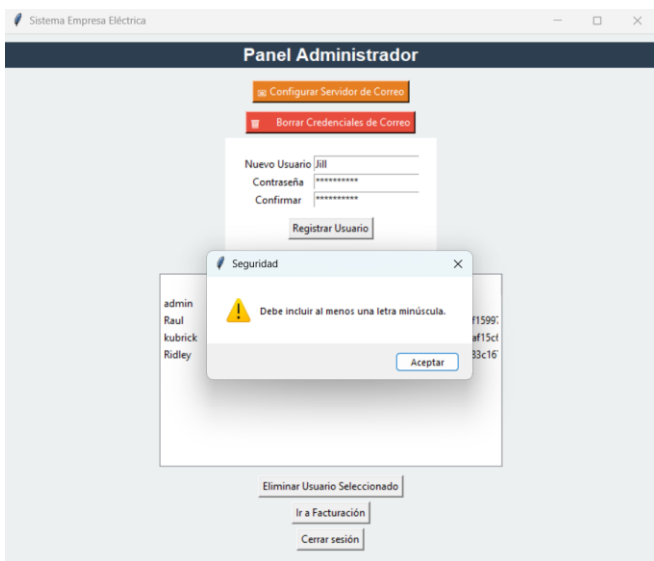
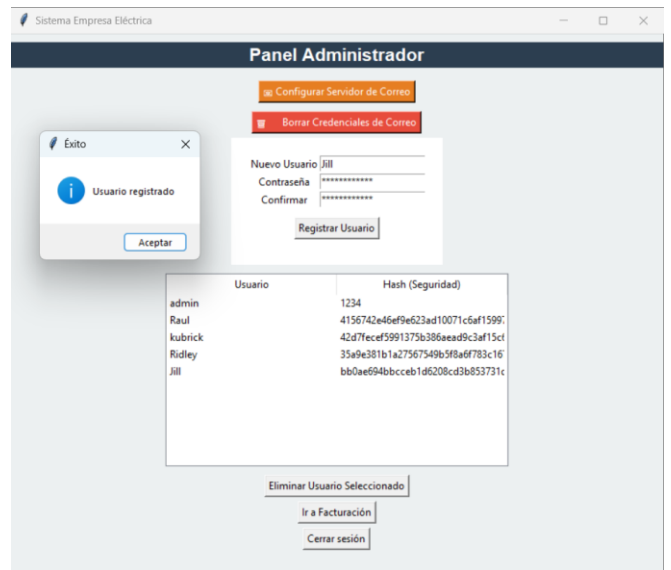
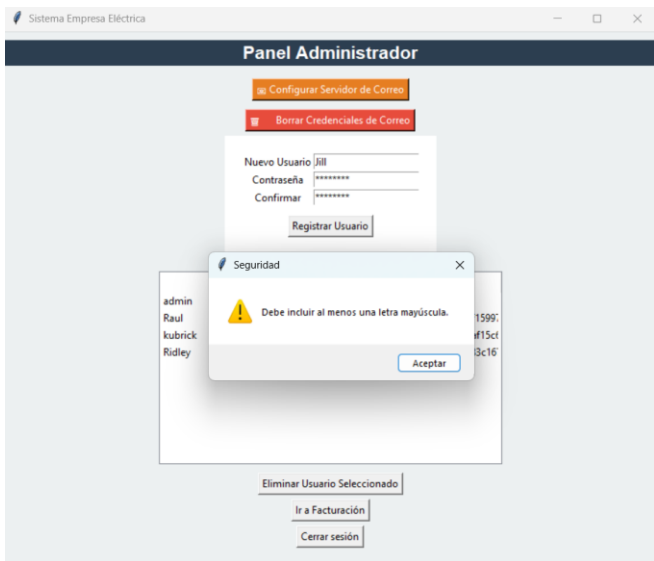
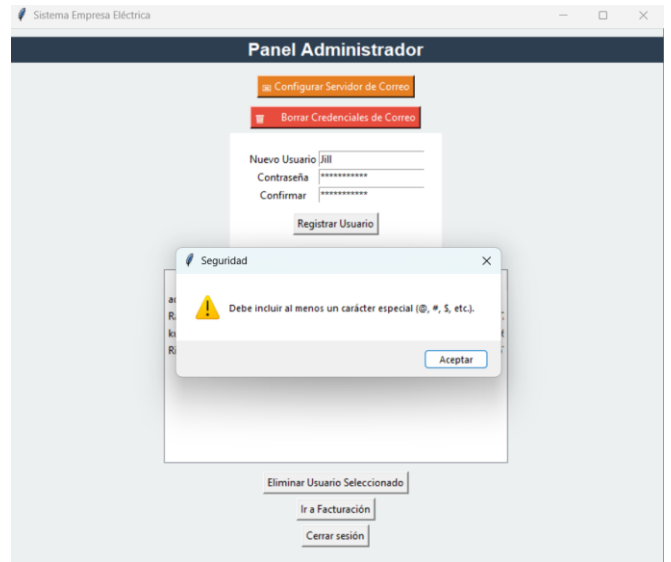
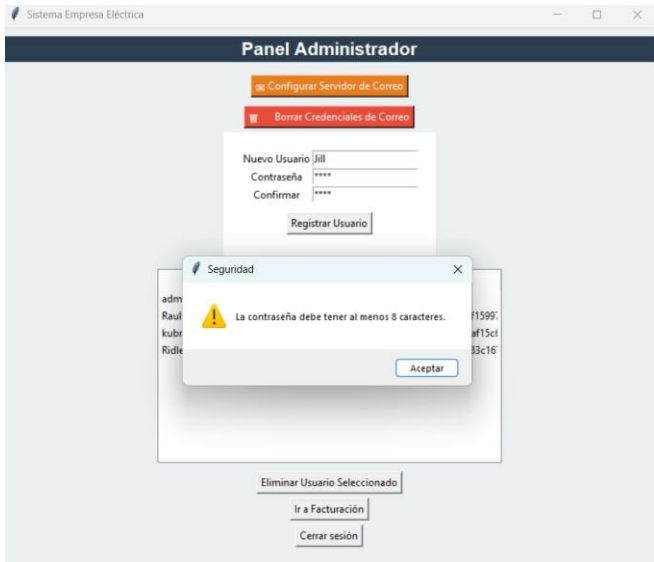
```

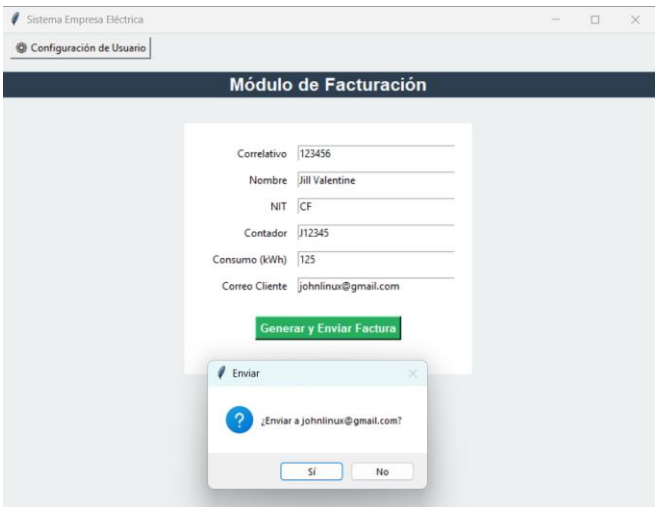
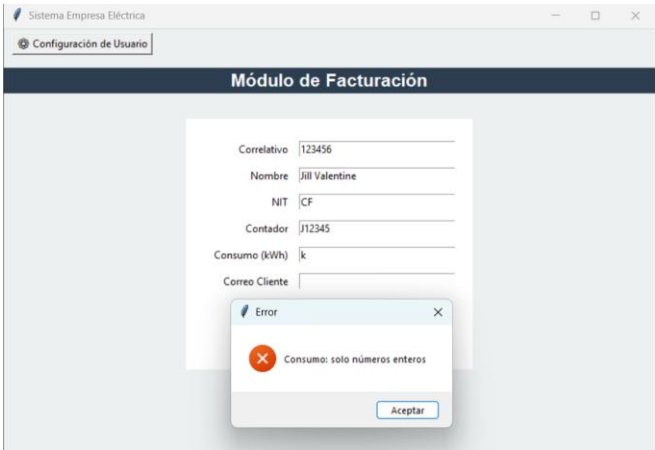
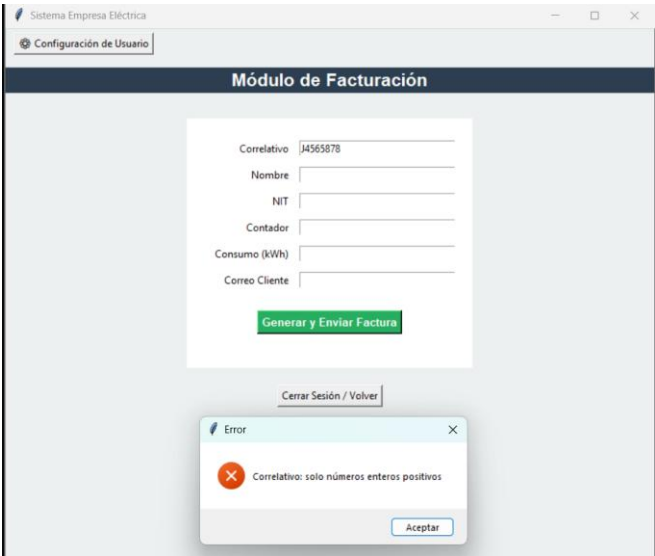
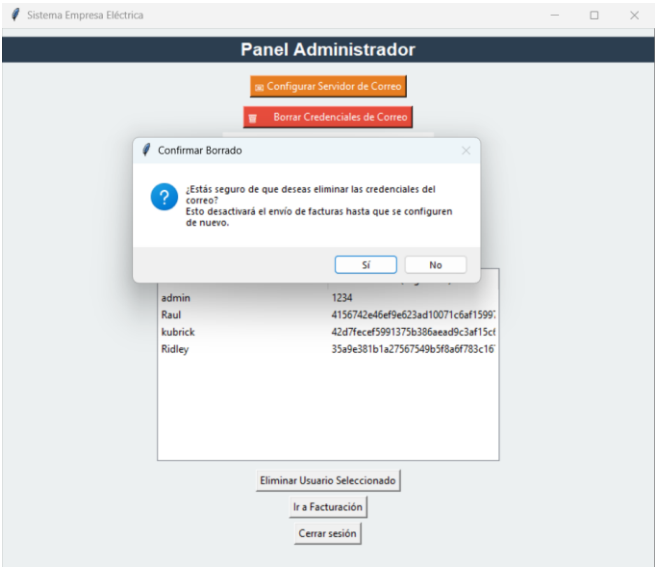
390         pdf.set_font("Arial", "B", 12)
391         pdf.cell(0, 5, "DATOS DEL CLIENTE", 0, 1)
392         pdf.set_font("Arial", "", 11)
393         pdf.cell(0, 5, f"Nombre: {nom}", 0, 1)
394         pdf.cell(0, 5, f"NIT: {nit}", 0, 1)
395         pdf.cell(0, 5, f"ID Contador: {cont}", 0, 1)
396         pdf.ln(10)
397
398         # Tabla
399         pdf.set_font("Arial", "B", 11)
400         pdf.cell(0, 5, "Descripción", 1); pdf.cell(40, 8, "Consumo", 1);
401         pdf.cell(40, 8, "Precio U.", 1); pdf.cell(40, 8, "Subtotal", 1); pdf.ln()
402         pdf.set_font("Arial", "", 11)
403         pdf.cell(0, 5, "Servicio Eléctrico", 1); pdf.cell(40, 8, f"Consumo kWh", 1); pdf.ln()
404         pdf.cell(40, 8, f"Q(precio_kwh)", 1); pdf.cell(40, 8, f"Q(subtotal:2f)", 1); pdf.ln()
405
406         pdf.ln(5)
407         pdf.cell(40, 8, f"IVA (12%)", 0, 0, "R"); pdf.cell(40, 8, f"Q(iva:2f)", 0, 1, "R");
408         pdf.set_font("Arial", "B", 10)
409         pdf.cell(40, 10, "TOTAL", 0, 0, "R"); pdf.cell(40, 10, f"Q(total:2f)", 0, 1, "R")
410
411         ruta = f"Facturas/factura_{corr}.pdf"
412         pdf.output(ruta)
413         registrar_bitacora(f"Factura {corr} creada para {nom}", self.usuario_actual)
414         messagebox.showinfo("Éxito", f"Factura generada en {ruta}")
415
416         if email_dest and "g" in email_dest:
417             if messagebox.askyesno("Enviar", f"¿Enviar a {email_dest}?"):
418                 if self.enviar_correo(pdf(ruta), email_dest, corr):
419                     messagebox.showinfo("Correo", "Enviado exitosamente")
420                 else:
421                     messagebox.showerror("Correo", "Fallo al enviar. Revise configuración SMTP.")
422
423
424
425     # ---- INICIO DE LA APLICACIÓN ----
426     def crear_directorio():
427         if not os.path.exists("configuracion/usuarios.txt"):
428             with open("configuracion/usuarios.txt", "w") as f:
429                 f.write(f"({admin}),{encryptar('Admin.2026')}\n") # Password inicial segura
430
431         root = Tk()
432         app = App(root)
433         root.mainloop()

```

VI. Resultado







EMPRESA ELÉCTRICA S.A.

Factura No: 123456

Fecha: 11/05/2026 11:08

DATOS DEL CLIENTE

Nombre: Jill Valentine

NIT: CF

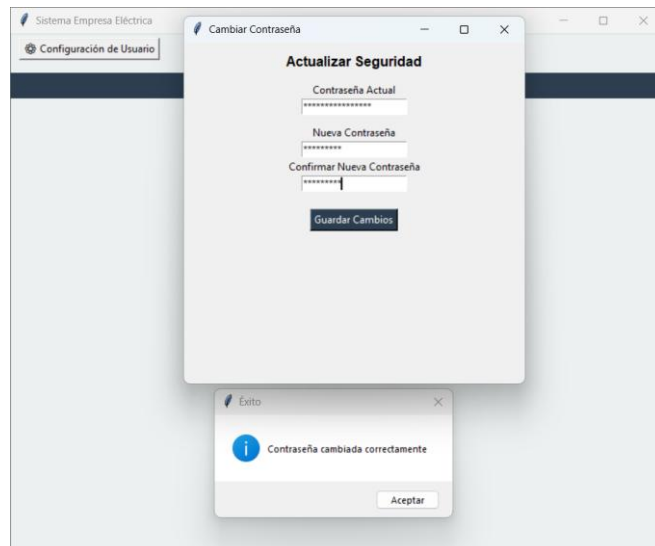
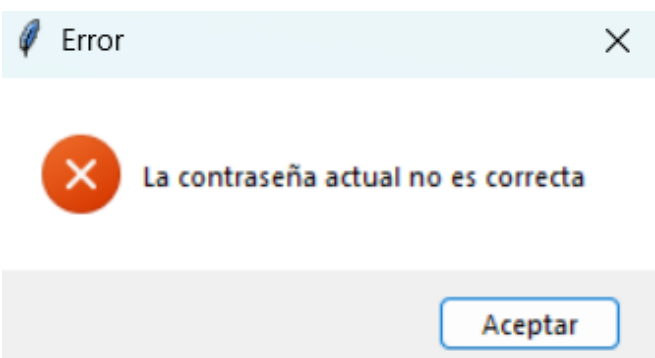
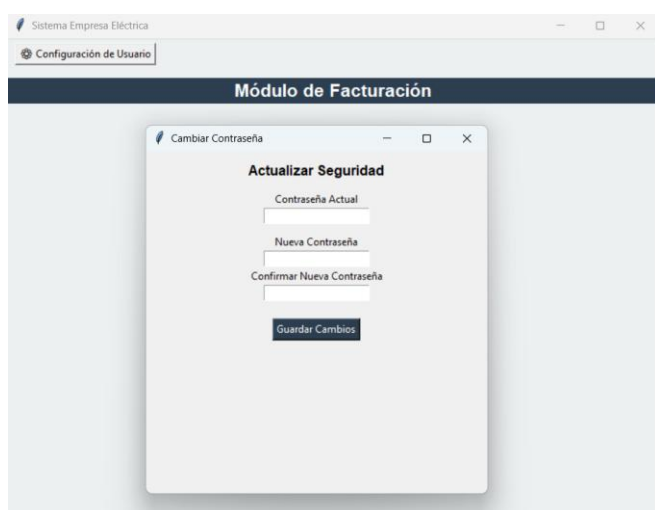
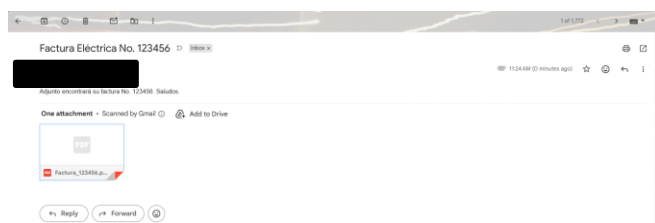
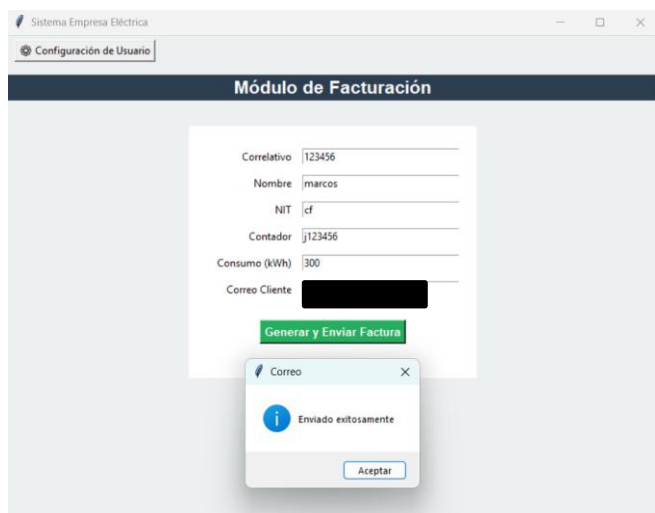
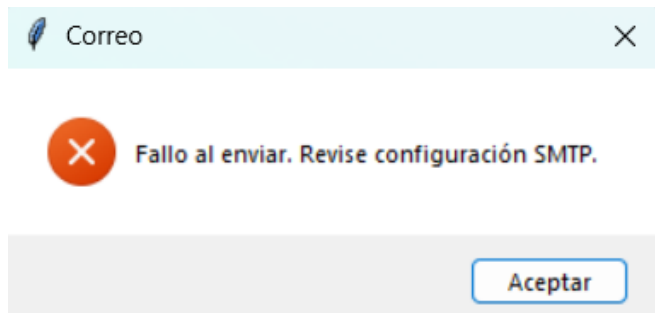
ID Contador: J12345

Descripción	Consumo	Precio U.	Subtotal
Servicio Eléctrico	125.0 kWh	Q1.42	Q177.50

IVA (12%): Q21.30

TOTAL: Q198.80

```
[2026-05-11 10:53:04] (admin) Inicio de sesión
[2026-05-11 11:01:04] (admin) Usuario creado: Jill
[2026-05-11 11:02:31] (admin) Configuración de correo eliminada
[2026-05-11 11:08:15] (admin) Factura 123456 creada para Jill Valentine
```

VII. Conclusiones

La organización de la interfaz gráfica en secciones claras y estructuradas permitió mejorar la experiencia del usuario, facilitando la navegación dentro del sistema y brindando un entorno visual más ordenado para la gestión de datos y generación de facturas.

La automatización del proceso de facturación mediante la generación de documentos PDF y su posterior envío por correo electrónico representa una mejora importante en la eficiencia del sistema, ya que reduce el trabajo manual y simula de una manera más realista el funcionamiento de una empresa eléctrica moderna.

La implementación de mecanismos de seguridad como el cifrado de contraseñas, validación de credenciales ayudó a fortalecer la protección de la información del sistema.

Finalmente, este proyecto permitió aplicar conocimientos de programación de manera práctica, desarrollando una solución completa que integra diferentes áreas de la informática y que puede servir como base para futuros sistemas administrativos más avanzados.

IV. Referencias

Mariano Reingart, Roman Kharin, Edward Ocasio, Marcelo Duarte, Yanone, Leo Zhu, Michael Kalbermatten, José L. Redrejo Rodríguez, Leonel Câmara y Craig Hobbs. *PyFPDF: Simple PDF generation for Python*. GitHub. Disponible en: [PyFPDF GitHub Repository](https://github.com/PyFPDF/PyFPDF)

